

Reentrancy Vulnerability Detection in Smart Contracts Based on Adversarial Examples

Fei Tong , Member, IEEE, Haoxuan Zang , Fang Jiang , Guodong Li , and Hongyi Qiu

Abstract—Reentrancy Vulnerabilities (RVs) in smart contracts pose severe financial risks to blockchain systems. While deep learning-based detection methods have shown promise, they suffer from critical limitations: (1) vulnerability to adversarial examples, (2) inadequate handling of Cross-Function RVs (CFRVs), and (3) reliance on limited training data. To address these challenges, we present SCAEG, the first adversarial example generation framework specifically designed for RV detection. SCAEG introduces five novel perturbation actions that preserve code functionality while exposing model vulnerabilities, including targeted modifications for CFRV scenarios. Our comprehensive evaluation demonstrates that SCAEG-generated examples significantly degrade the performance of state-of-the-art detectors (e.g., reducing AWDNN's accuracy by 27.51 percentage points), exposing their fragility to adversarial attacks. Building on these insights, we propose VoteRD, an ensemble detection model that combines semantic, rule-based, and positional features through a multi-model voting mechanism. Experimental results show VoteRD achieves superior robustness, outperforming existing approaches by 3.7% in recall and 7.4% in F1-score (compare with AWDNN), while effectively detecting both traditional RVs and CFRVs. This work not only advances RV detection capabilities but also establishes a new benchmark for evaluating model robustness in smart contract security analysis. SCAEG and VoteRD form a synergistic framework: SCAEG systematically evaluates model robustness, while VoteRD leverages adversarial insights to enhance detection accuracy. Our datasets and tools are publicly released to support future research in adversarial testing and vulnerability detection.

Index Terms—Smart contract, reentrancy vulnerability, adversarial example, cross-function.

I. INTRODUCTION

SMART contracts [1] have been widely adopted in business and financial applications due to their characteristics of automatic execution and public traceability. As these contracts are predominantly deployed on permissionless blockchains, their security has garnered significant attention, with poorly designed

contracts potentially exposing vulnerabilities to malicious actors [2].

Reentrancy vulnerabilities have emerged as one of the most destructive security risks in modern smart contract ecosystems, with quantifiable economic impact. Since the infamous DAO reentrancy attack in 2016, which resulted in a \$150 million loss in digital assets [3], reentrancy attacks have continued to occur in real-world deployments. In the third quarter of 2024 alone, reentrancy-related security incidents caused \$93.46 million in financial losses [4], indicating that the threat has not subsided despite improved developer awareness and maturing security practices. Entering 2025, industry security reports further reveal that smart contract vulnerabilities collectively contributed to more than \$263 million in losses in the first half of the year [5]. Although these reports do not break down losses by specific vulnerability type, reentrancy, which is one of the most frequently exploited categories of smart contract flaws, remains a central component of this risk landscape.

While classic reentrancy patterns have become less prevalent due to increased developer awareness, modern variants continue to evolve and pose significant threats. Cross-Function RV (CFRV) and Read-Only RV (RORV) represent sophisticated adaptations that bypass conventional detection heuristics. This evolution necessitates robust detection frameworks capable of handling both known and emerging reentrancy manifestations, rather than relying on pattern-matching of historical vulnerabilities.

While the aforementioned incidents exploit vulnerabilities in contract logic, an emerging and equally critical threat targets the *vulnerability detection systems* themselves. As deep learning-based detectors become increasingly adopted for smart contract auditing, their robustness against adversarial manipulation becomes paramount. Our experimental evaluation reveals the severity of this threat: adversarial transformations can achieve success rates of up to 45.3% against state-of-the-art detectors (Table III), effectively allowing malicious contracts to evade detection. Furthermore, cross-dataset evaluation demonstrates performance degradation of over 25 percentage points in F1-score when detectors face adversarial examples (Fig. 13). This vulnerability in the detection infrastructure could lead to a false sense of security, where contracts passing automated scans may still contain hidden vulnerabilities.

Since smart contracts are immutable once deployed on the blockchain, thorough security audits prior to deployment are essential. Traditional vulnerability detection approaches, including symbolic execution [6], [7], [8] and taint analysis [6],

Received 16 December 2025; revised 8 May 2026; accepted 19 May 2026. Date of publication 25 May 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62472088 and Grant 625B2047 and in part by the Big Data Computing Center of Southeast University. (Corresponding author: Fei Tong.)

Fei Tong is with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China, also with the State Key Laboratory of Internet Architecture, Tsinghua University, Beijing 100084, China, and also with the Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing 211103, China (e-mail: ftong@seu.edu.cn).

Haoxuan Zang, Fang Jiang, Guodong Li, and Hongyi Qiu are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: 213220612@seu.edu.cn; 230239271@seu.edu.cn; 213222120@seu.edu.cn; 1742437018@qq.com).

Digital Object Identifier 10.1109/TDSC.2026.3696629

[9], [10], have been widely employed in practice. However, these methods typically rely on expert-defined rules and handcrafted patterns, which are time-consuming to develop and maintain. Furthermore, such rule-based techniques often suffer from high false-positive rates [11], diminishing their practical utility. As the number and complexity of smart contracts continue to grow exponentially, manually updating and scaling these detection rules becomes increasingly challenging, thereby limiting the adaptability and efficiency of traditional detection systems. More fundamentally, program-analysis-based approaches, including symbolic execution [7], [8] and taint/fuzzing analysis [9], [10], are hampered by inherent limitations such as path explosion, incomplete execution coverage, and inadequate modeling of evolving smart contract semantics. These constraints ultimately hinder their scalability and robustness when confronted with adversarially crafted contracts, motivating the need for more adaptive, data-driven detection mechanisms.

The emergence of artificial intelligence has opened new avenues for automating vulnerability detection in smart contracts, promising enhanced efficiency and accuracy. In recent years, deep learning techniques have been increasingly applied to source code analysis tasks, including smart contract vulnerability detection. However, a critical examination of current research and available datasets reveals a fundamental limitation: the scarcity of high-quality, publicly accessible datasets specifically targeting smart contract vulnerabilities. Most existing approaches [12], [13] evaluate their models exclusively on self-constructed datasets, which compromises the generalizability and reproducibility of their findings. Moreover, insufficient attention has been paid to model robustness, particularly in adversarial settings. Consequently, these models remain vulnerable to adversarial example attacks, where minor input perturbations can lead to significant prediction errors. Additionally, current model-based approaches typically overlook CFRVs (a critical RV variant), and to date, no public datasets support effective training or evaluation for this class of vulnerabilities, further impeding progress in this domain [14], [15], [16], [17].

While adversarial example research has predominantly focused on functional classification tasks in traditional software, adversarial attacks specifically targeting smart contract RV detection models remain largely unexplored. To bridge this gap, we propose the Smart Contract Adversarial Example Generator (SCAEG), a black-box adversarial attack framework specifically designed for RV detection tasks. SCAEG generates labeled adversarial examples by applying five novel adversarial actions (singly and in combination) to the original dataset, while preserving the code's original structure and functionality. These samples are then evaluated against target models, with misclassified examples being incorporated into the dataset. Building upon this foundation, we further propose VoteRD, a Vote-based Reentrancy Vulnerability Detector. During preprocessing, VoteRD extracts three distinct feature types relevant to RVs (including CFRVs): semantic features, rule-based features, and positional features. Different models are strategically employed to handle these feature types according to their respective processing capabilities.

The main contributions of this article are summarized as follows:

- This paper introduces SCAEG, the first adversarial framework designed to systematically assess the robustness of RV detection models in smart contracts.
- The work specifically addresses the overlooked challenge of CFRVs. The integration of CFRV patterns into SCAEG's generation process uniquely reveals model deficiencies in handling inter-procedural attack vectors.
- Furthermore, the paper proposes VoteRD, a novel ensemble model informed by the adversarial analysis. VoteRD is capable of detecting both conventional RVs and CFRVs, and achieves significant performance gains, with a 3.7% increase in recall and a 7.4% improvement in F1-score over state-of-the-art baselines.

The remainder of this paper is organized as follows. Section II reviews related work. Section III provides necessary preliminaries. The detailed designs of SCAEG and VoteRD are presented in Sections IV and V, respectively. Section VI conducts comprehensive experiments to evaluate both SCAEG and VoteRD. Finally, Section VII concludes the paper.

II. RELATED WORK

This section reviews the landscape of smart contract RV detection, positioning our work at the intersection of deep learning-based approaches and adversarial robustness. We first dissect the evolution of RV detection methods, highlighting a paradigm shift from traditional static analysis to data-driven deep learning techniques.

Traditional static analysis methods [6], [9], [18] rely on manually crafted rules to identify vulnerability patterns. While effective to a degree, these methods suffer from several inherent limitations: (1) they often produce high false-positive rates due to the inability to comprehend complex code semantics, (2) scaling and maintaining an exhaustive set of rules for evolving contract codebases is challenging and labor-intensive, and (3) they struggle with path explosion and often achieve incomplete code coverage, leaving subtle vulnerabilities undetected. These limitations motivate the need for more adaptive and scalable detection mechanisms.

To structure this discussion, this section is organized into two main subsections. Section II-A surveys *RV Detection Based on Deep Learning*, categorizing existing approaches and identifying their common limitations regarding generalization and robustness. Subsequently, Section II-B explores *Adversarial Example Attack in RV Detection*, reviewing relevant work in software security and underscoring the distinct gap our method, SCAEG, aims to fill.

A. RV Detection Based on Deep Learning

Numerous tools have been developed to detect smart contract RVs. Traditional static analysis methods [6], [9], [18] rely on manually crafted rules to identify vulnerability patterns. While effective to a degree, these methods suffer from several inherent limitations: (1) they often produce high false-positive rates due to the inability to comprehend complex code semantics, (2)

scaling and maintaining an exhaustive set of rules for evolving contract codebases is challenging and labor-intensive, and (3) they struggle with path explosion and often achieve incomplete code coverage, leaving subtle vulnerabilities undetected.

To overcome these limitations, deep learning approaches [12], [13], [19], [20] have emerged, offering a data-driven alternative that does not depend on pre-defined rules. These methods primarily focus on improving detection accuracy and can be broadly classified into three categories. The first and most prevalent category treats source code or bytecode as specialized textual data. By leveraging natural language processing techniques, these methods transform vulnerability detection into a text classification task. For instance, Peng et al. [21] processed smart contract source code as textual input. The methodology involves splitting code fragments into serialized tokens through lexical analysis, converting them into vector representations using Word2Vec [22], and subsequently feeding these vectors into Long Short-Term Memory (LSTM) networks [23] for prediction. Similarly, VASCOT [24] models contracts as opcode sequences and employs a Transformer with a sliding-window mechanism, showing that low-level sequential representations can also be effective, especially when source code is unavailable. Despite these advances, sequence-based methods in general capture limited vulnerability characteristics while retaining substantial redundant information. Furthermore, since these models primarily extract features from token or opcode sequences, their ability to capture the code's control flow and function call relationships is often insufficient. To address these limitations and effectively integrate local pattern recognition with global semantic modeling, Zhang et al. [25] integrated Bidirectional Gated Recurrent Units (BiGRU), attention mechanisms, and Support Vector Machines (SVM) in a hybrid architecture.

The second category represents smart contracts as graph structures, typically leveraging program-specific constructs such as control-flow and data-flow graphs. To extract RV-relevant features from source code, Zhang et al. [19] modeled smart contract code as contract graphs based on data and control dependencies. The authors proposed two graph neural networks for processing: Degree-Free Graph Convolutional Networks (DR-GCN) and temporal edge transfer neural networks. Experimental evaluations demonstrate promising performance for these models. However, the process of abstracting contract code into graph representations introduces significant complexity, resulting in substantial computational overhead. Subsequent studies further explored the integration of structural and semantic features. Liu et al. [26] extended Zhuang et al.'s approach by incorporating additional pattern features, yet these supplementary features remain overly simplistic and fail to adequately capture characteristics related to reentrancy mitigation mechanisms. Building upon this work, Liu et al. [27] attempted to enhance interpretability by extracting local patterns, constructing local graphs, and processing both through attentive multi-encoder networks to identify salient characteristics. Nevertheless, the interpretability provided by Liu et al. [27] remains relatively coarse and offers limited transparency into the model's internal decision-making processes.

More recent works have emphasized richer feature integration and semantic modeling. For example, Guo et al. [28] combines bytecode-level control-flow graphs with expert-designed source-level patterns and introduces subspace mapping strategies to address feature heterogeneity, demonstrating the complementary strengths of graph structures and rule-based knowledge. ByteEye [29] enhances bytecode-level CFGs with edge augmentation and incorporates both general and vulnerability-specific features, including security mechanism indicators, achieving higher F1 scores and demonstrating effectiveness in identifying real-world vulnerabilities. These studies collectively suggest that relying solely on semantic learning or rule-based features is insufficient under distribution shifts, while multi-feature fusion provides a more robust solution aligned with the underlying mechanisms of reentrancy. This insight directly motivates our design of VoteRD, which fuses semantic, rule-based, and positional features through a multi-model ensemble to achieve complementary robustness.

Compared with sequence-based models, graph-based models are capable of enhancing a model's ability to model cross-function calls and complex control paths, thereby more accurately capturing the typical semantic features of RVs. However, these methods still face two major challenges: 1. High Construction Cost: Both the graph building cost and the training efficiency are significantly lower than those of sequence-based neural network models. 2. Poor Robustness: graph-based models are susceptible to code-level perturbations, resulting in relatively poor modeling robustness.

The third category focuses on specialized feature representations that explicitly encode reentrancy-related security semantics into the detection process. Unlike generic sequence or graph-based methods, these approaches emphasize modeling domain-specific characteristics such as cross-function dependencies, protection mechanisms, and attack execution patterns. AutoAR [30] introduces a dedicated program dependence graph (RentPDG) to preserve reentrancy-related control and data dependencies, and employs graph autoencoders with clustering to identify common anti-reentrancy patterns in an unsupervised manner. Notably, it is designed to be integrated into existing detection pipelines and demonstrates a significant reduction in false positives, providing strong evidence for the value of explicitly modeling defense mechanisms as features. SmartReco [31] further incorporates cross-DApp boundary identification, entry function localization, and on-chain data-driven validation, highlighting that missing contextual information is a key factor in emerging reentrancy variants such as read-only reentrancy. AdvScanner [32] takes a complementary adversarial perspective by modeling exploit generation as constrained program synthesis, forming a closed loop between vulnerability detection and exploit validation. While AdvScanner focuses on generating exploitable contracts to assess detectors, our SCAEG framework targets a distinct goal: systematically exposing model blind spots through semantics-preserving perturbations without constructing full exploits.

These studies indicate that explicitly incorporating semantic features related to reentrancy mechanisms can effectively improve detection reliability. However, approaches such as

AutoAR and SmartReco often rely on carefully designed features or domain knowledge as the primary detection mechanism, which may limit their generalization under diverse contract patterns. In contrast, VoteRD adopts a hybrid strategy in which domain-specific features (e.g., rule-based and positional features) serve as auxiliary signals within a predominantly data-driven ensemble, thereby retaining the precision benefits of domain knowledge while remaining adaptable through learned semantic representations.

Furthermore, the absence of unified datasets for smart contract RV detection presents a significant challenge. Existing studies typically evaluate models on self-constructed datasets, potentially leading to overfitting and limited robustness. Moreover, most available datasets lack representative examples of CFRVs, a critical yet frequently overlooked RV variant. Consequently, models trained on such limited datasets remain vulnerable to adversarial example attacks and demonstrate inadequate performance in detecting CFRVs in practical scenarios. Finally, we note that the works reviewed in this section focus on code-level vulnerability detection. Complementary approaches, such as transaction-level analysis for malicious account and scam detection [33], address different attack surfaces and fall outside the scope of this survey.

B. Adversarial Example Attack in RV Detection

Adversarial example generation has emerged as a vital research direction for evaluating model robustness and interpretability, yet its application to vulnerability detection in smart contracts remains notably limited.

The concept of adversarial examples originated from image classification tasks in deep learning [34]. By constructing and analyzing adversarial examples, researchers can identify model weaknesses and incorporate adversarial training to enhance robustness. Simultaneously, examining adversarial examples provides insights into model internal mechanisms, thereby contributing to improved interpretability.

Zhang et al. [35] focused on C++ function classification models, employing a Metropolis-Hastings modifier approach to generate adversarial examples through variable name substitutions. However, this transformation strategy proves overly simplistic, and the iterative process incurs significant computational costs. Moreover, its effectiveness remains limited in vulnerability detection scenarios where variable names are typically normalized. Building on the limitations of such simple transformations, Tian et al. [36] designed more transformative methods using Q-learning-based Markov decision processes, demonstrating improved performance compared to Zhang et al. [35]. Nevertheless, their approach remains heavily constrained by programming language specifics and proves unsuitable for smart contract languages. Ferretti et al. [37] utilized grammatical evolution for adversarial example generation, but this method also involves iterative search processes with substantial time requirements. In contrast Jha et al. [38] modified original code semantics to strengthen attack capability, an approach incompatible with our adversarial example scenario that requires functional preservation. Yang et al. [39] developed the CodeTAE attack, demonstrating the feasibility of generating cross-domain

adversarial examples using genetic algorithms, yet their work does not specifically target smart contracts, leaving a gap in the context of contract-level vulnerability detection.

Notably, adversarial example attacks targeting RV detection models remain severely underexplored, and existing studies offer no unified definition of adversarial action in smart-contract security. Nonetheless, recent studies started to reveal a list of concrete adversarial behaviors that can harm the integrity of vulnerability detection systems. Wu et al. [32] showed that semantics-preserving transformations can be used for constructing adversarial smart contracts that incorporate reentrancy attacks, callback flows, and exploit routes, remaining syntactically valid and compiled, while concealing important execution semantics from analysis. Besides, analysis of adversarial DeFi contracts [40] with actions separate common action patterns in these models, including concealed fund sources, distinctive call-flow structures, and frequent token-related interactions. These actions ideally differ from harmless models and help identify how attackers transform normal behavior patterns. These studies confirm that various actions are being developed for potential manipulation in smart contracts. Simultaneously, designs of actions for programming languages are unsatisfactory for smart-contract environments [36] since all transformations in models are supposed to preserve EVM syntax, as well as determinism and functional constraints.

Beyond dedicated adversarial generation tools, recent advances in large language models (LLMs) have opened new possibilities for smart contract analysis. AdvScanner [32] leverages LLMs for automated exploit generation, demonstrating that LLMs can effectively reason about contract semantics and construct adversarial scenarios. However, in the context of vulnerability detection rather than exploit generation, directly applying general-purpose LLMs faces several challenges. First, the limited scale and diversity of existing reentrancy datasets [13], [21] make it difficult to reliably fine-tune LLMs without overfitting. Second, LLMs are not explicitly designed to capture vulnerability-specific execution patterns such as cross-function call dependencies and state-update ordering, which are critical for detecting CFRVs. Third, the opaque decision process of LLMs poses interpretability challenges for security-critical auditing. These considerations motivate the design of specialized, feature-rich ensemble models that can effectively operate under constrained data and explicitly encode domain-specific security knowledge.

III. PRELIMINARY

This section establishes the foundational knowledge required to understand our work. Section III-A formally defines smart contract RVs, illustrating the attack mechanism with a canonical example. Subsequently, Section III-B introduces the concept of adversarial examples in the context of code models, defining the key terminology and formulation that underpin our adversarial generation framework, SCAEG.

A. Smart Contract RVs

RVs in smart contracts arise from improper programming practices that violate the checks-effects-interactions pattern. A

```

pragma solidity ^0.4.21;
Contract Bank{
    .....
    function withdraw() public
        require(balances[msg.sender] >= 0);
        require(msg.sender.call.value(balances[msg.sender])());
        balances[msg.sender] = 0; }
    .....
}

```

Fig. 1. Classic RV example (simplified for illustrative purposes).

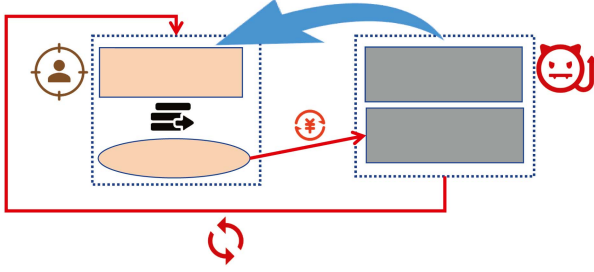


Fig. 2. Exploitation process of the RV depicted in Fig. 1.

typical RV example is illustrated in Fig. 1, with its exploitation process detailed in Fig. 2. Consider an attack scenario where an adversary initially invokes the `withdraw` function in a Bank contract to execute a withdrawal operation. The `withdraw` function first validates the user's balance, then performs an external call to transfer funds to the user. Critically, the contract has not yet updated the user's balance to zero at this stage, meaning the Bank contract still recognizes the attacker as possessing withdrawable funds. When the attacker's contract receives the initial refund, it automatically triggers its `fallback` function, which recursively calls the `withdraw` function again. This process, depicted by the red arrows in Fig. 2, establishes a malicious loop: `attack` → `withdraw` → `fallback` → ... → `withdraw` → `fallback`. The attacker can continuously drain funds from the Bank contract because the balance reset operation is positioned after the external call in the `withdraw` function, allowing it to be consistently bypassed during reentrant executions.

While Figs. 1 and 2 focus on the canonical form of reentrancy for conceptual clarity, subsequent Section IV-B introduce more realistic RV variants—such as cross-function reentrancy (CFRV) and read-only reentrancy (RORV)—to reflect the broader attack surface observed in modern smart-contract systems.

B. Adversarial Examples

Vulnerability detection represents a crucial and widely studied task with significant implications for preventing substantial economic losses. Formally, a labeled dataset for this task is defined as:

$$D = (X, Y), \quad (1)$$

where X denotes a collection of smart contract code snippets, with each $x \in X$ representing an intermediate code representation (e.g., character sequences or token sequences). Y constitutes

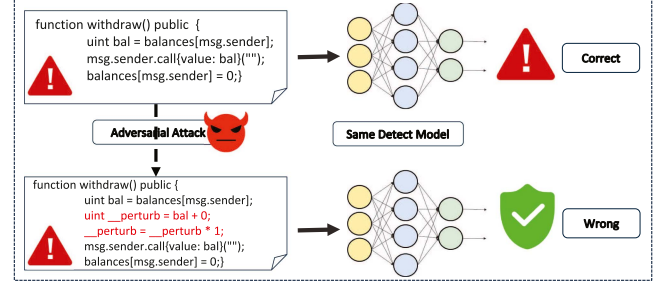


Fig. 3. Illustration of an adversarial example with inserted redundant code.

the corresponding label set, where each $y \in Y$ represents the vulnerability label for sample x . The prediction generated by model M is expressed as $y_{predict} = M(x)$. When subtle perturbations $\Delta(x)$ are applied to x , satisfying $\Delta(x) \in \Theta$:

$$x_{adv} = x + \Delta(x), \quad \Delta(x) \in \Theta, \quad (2)$$

the resulting prediction $y_{adv} = M(x_{adv})$ may substantially deviate from the original prediction. The perturbation $\Delta(x)$ must adhere to specific constraints Θ to ensure the modified sample remains functionally equivalent while syntactically valid. Such manipulated samples are termed adversarial examples. Fig. 3 presents a simple adversarial example where RV-related redundant code (highlighted in red) is inserted. Empirical evidence demonstrates that even these subtle perturbations, easily distinguishable by human analysts, can significantly mislead model predictions.

IV. DESIGN OF SCAEG

This section details the design of our SCAEG. We begin in Section IV-A by presenting the overall workflow and the core algorithm for generating adversarial examples. Section IV-B then introduces the five novel adversarial actions that form the backbone of SCAEG's perturbation strategy, complete with illustrative examples. Finally, Section IV-C describes the grammar verification process that ensures the functional validity of all generated samples.

Existing adversarial example generation methods for source code analysis typically rely on token replacement [38] or code insertion [35]. However, these approaches prove ineffective in the context of RV detection, necessitating the design of specialized adversarial actions tailored to this domain.

Given that the vast majority of smart contracts are developed in Solidity, particularly on the Ethereum platform, and considering Solidity's dominance as the most widely adopted smart contract programming language, our framework specifically targets Solidity-based contracts to ensure practical applicability and effectiveness.

A. Workflow

The objective of SCAEG is to generate adversarial examples that preserve the original vulnerability labels. For instance, vulnerable samples should remain vulnerable after transformation, while non-vulnerable samples should maintain their

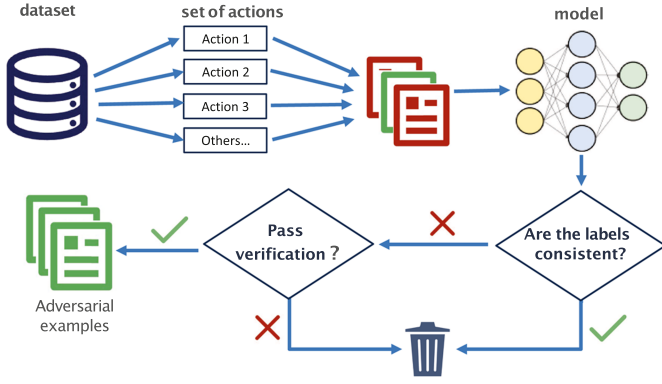


Fig. 4. Workflow of SCAEG.

Algorithm 1: Adversarial Example Generation.

Input: Dataset X , Label Y , Target Model M , Set of Actions S
Output: Set of adversarial examples X_{adv}

- 1: $X_{adv} \leftarrow \emptyset$
- 2: **for each** $x_i \in X$ **do**
- 3: Input x_i to M and obtain prediction y_{pred}
- 4: **if** y_{pred} is consistent with the ground truth label **then**
- 5: **for each** $Action_j$ in S **do**
- 6: Apply $Action_j$ on x_i to generate $x_{Action}^{(j)}$
- 7: Input $x_{Action}^{(j)}$ to M and obtain prediction $y_{pred}^{(j)}$
- 8: **if** $y_{pred}^{(j)} \neq y_{pred}$ and $x_{Action}^{(j)}$ passes compilation **then**
- 9: Add $x_{Action}^{(j)}$ to X_{adv}
- 10: **end if**
- 11: **end for**
- 12: **end if**
- 13: **end for**
- 14: **return** X_{adv}

non-vulnerable status. This process constitutes an adversarial attack aimed at creating misleading inputs for detection models.

As illustrated in Fig. 4, SCAEG operates by first identifying samples that are correctly classified by the target model. It then iteratively applies designed adversarial actions to modify these samples. The transformed samples are subsequently fed back into the model for reevaluation. Those samples that both induce misclassification and pass grammatical validation are retained as successful adversarial examples, thereby exposing model vulnerabilities. The complete algorithmic procedure is detailed in Algorithm 1.

B. Adversarial Actions

SCAEG's adversarial actions are designed to be pattern-agnostic, targeting fundamental weaknesses in detection models rather than specific vulnerability instances. This ensures relevance across evolving reentrancy variants.

TABLE I
ADVERSARIAL ACTIONS IN SCAEG

Action	Description
Cross-function Call	Transform <code>.call.value</code> operations into cross-function calls
Fake Modifier	Insert ineffective modifiers that provide no actual protection
Redundant Code	Add semantically irrelevant code segments that appear vulnerability-related
Multiple Functions	Split withdrawal operations across multiple <code>.call.value</code> instructions
Read-Only Hijack	Encapsulate <code>.call.value</code> logic within view functions returning boolean values

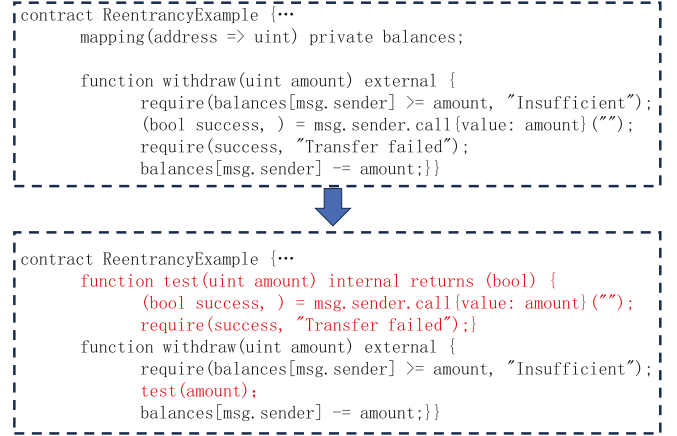


Fig. 5. First-type CFRV example.

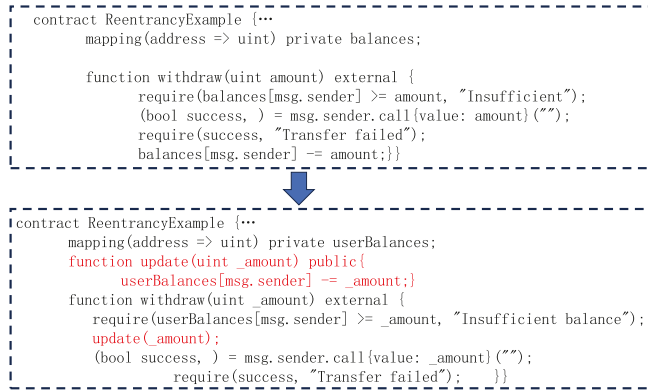
Through comprehensive analysis of reentrancy vulnerability principles, we design five adversarial actions as summarized in Table I.

Action 1. Cross-function Call: Existing detection methods predominantly focus on functions containing `call` operations while overlooking caller-callee relationships. This limitation renders CFRVs particularly challenging for current detectors. Due to the scarcity of CFRV samples in existing datasets, we systematically construct such examples through automated transformation.

We categorize CFRVs into two distinct types. The first type, illustrated in Fig. 5, encapsulates low-level `call` operations within separate internal functions (e.g., `test`). Determining vulnerability presence requires analyzing both the callee function and its calling context (e.g., `withdraw`), as examination of either component in isolation proves insufficient. This encapsulation strategy effectively obscures vulnerability analysis, presenting significant challenges for detection models.

The second CFRV type arises when multiple functions share critical state variables, with at least one function performing unsafe updates. As shown in Fig. 6, the `withdraw` function includes a balance validation check via `require`, yet remains vulnerable because an attacker can invoke the `update` function before balance deduction occurs. This enables not only legitimate balance retrieval but also unauthorized fund extraction through state manipulation.

While both CFRV types preserve original vulnerability labels, the first type demonstrates broader applicability, affecting all



```

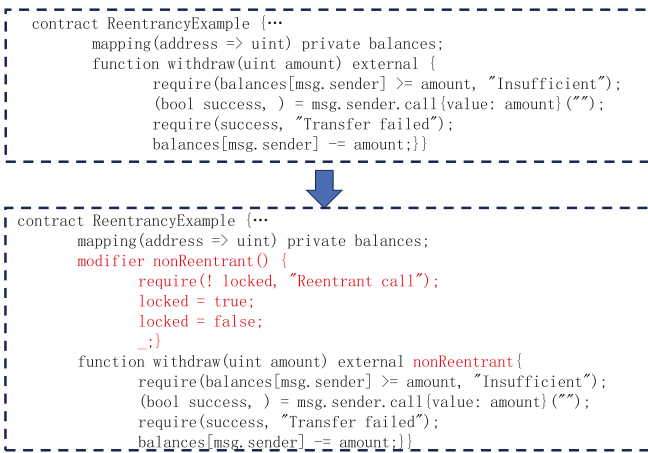
contract ReentrancyExample {...
    mapping(address => uint) private balances;

    function withdraw(uint amount) external {
        require(balances[msg.sender] >= amount, "Insufficient");
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
        balances[msg.sender] -= amount;
    }
}

contract ReentrancyExample {...
    mapping(address => uint) private userBalances;
    function update(uint _amount) public{
        userBalances[msg.sender] -= _amount;
    }
    function withdraw(uint _amount) external {
        require(userBalances[msg.sender] >= _amount, "Insufficient balance");
        update(_amount);
        (bool success, ) = msg.sender.call{value: _amount}("");
        require(success, "Transfer failed");
    }
}

```

Fig. 6. Second-type CFRV example.



```

contract ReentrancyExample {...
    mapping(address => uint) private balances;
    function withdraw(uint amount) external {
        require(balances[msg.sender] >= amount, "Insufficient");
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
        balances[msg.sender] -= amount;
    }
}

contract ReentrancyExample {...
    mapping(address => uint) private balances;
    modifier nonReentrant() {
        require(! locked, "Reentrant call");
        locked = true;
        locked = false;
        _;
    }
    function withdraw(uint amount) external nonReentrant{
        require(balances[msg.sender] >= amount, "Insufficient");
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
        balances[msg.sender] -= amount;
    }
}

```

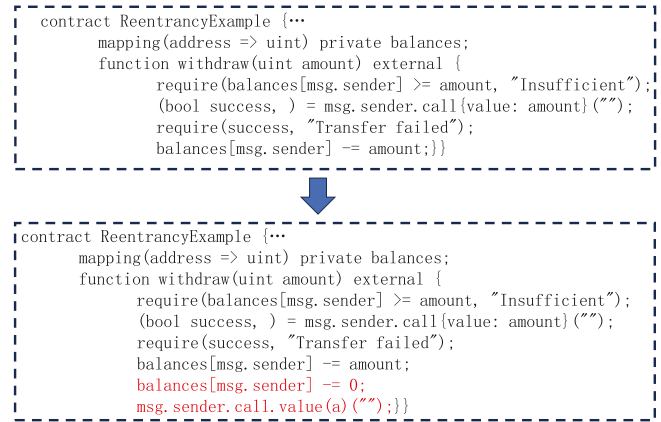
Fig. 7. Fake modifier example.

contracts employing `call` instructions. The second type relies on specific contextual patterns that resist generalization through unified templates. Consequently, Action 1 implements transformations based on the first CFRV type.

The transformation process begins by using regular expressions to identify `.call` statements within input files. For each detected statement, the framework parses the Solidity Abstract Syntax Tree (AST) to locate the enclosing function and contract. The contract content is then analyzed to extract caller information and parameters from the `.call` statement. Based on parameter count and structure, a new private `test` function is generated to encapsulate the call behavior. The original `.call` statement is replaced with an invocation of this new function, which is subsequently integrated into the contract source code.

Action 2. Fake Modifier: In Solidity, modifiers represent key language constructs for enforcing pre- and post-conditions on function execution. Existing detection approaches typically examine modifier names in function declarations but neglect to analyze modifier implementation details, leading to high false positive rates. Incorrectly implemented modifiers may fail to provide intended protection while creating a false sense of security.

Action 2 addresses this limitation by inserting ineffective modifiers that appear protective but provide no actual security



```

contract ReentrancyExample {...
    mapping(address => uint) private balances;
    function withdraw(uint amount) external {
        require(balances[msg.sender] >= amount, "Insufficient");
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
        balances[msg.sender] -= amount;
    }
}

contract ReentrancyExample {...
    mapping(address => uint) private balances;
    function withdraw(uint amount) external {
        require(balances[msg.sender] >= amount, "Insufficient");
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
        balances[msg.sender] -= amount;
        balances[msg.sender] -= 0;
        msg.sender.call.value(a)("");
    }
}

```

Fig. 8. Redundant code example.

guarantees. As illustrated in Fig. 7, these fake modifiers maintain original vulnerability labels while potentially misleading detection models that rely on superficial modifier presence.

The implementation process mirrors Action 1's initial steps for detecting `.call` statements and identifying their context. Once the target function is located, pre-designed fake modifiers are inserted preceding the function declaration, preserving code compilability while introducing deceptive syntactic patterns.

Action 3. Redundant Code: Code preprocessing represents a critical component in deep learning-based detection pipelines. Inadequate preprocessing can introduce redundancies that distract models toward irrelevant features. Action 3 inserts code segments that undergo processing but do not affect program semantics, such as the operations `balances[msg.sender] -= 0;` and `msg.sender.call.value(a)("")` shown in Fig. 8.

These redundant operations simulate additional state manipulations and external interactions without altering core withdrawal logic. They serve as distractors for static analysis tools and preprocessing routines, introducing noise that may mislead models while preserving functional behavior.

The transformation process identifies `.call` statements and appends predefined redundant code segments after the final line of the containing contract, ensuring that the additions do not interfere with existing logic.

Action 4. Multiple Functions: We observe that some detection approaches [41], [42] terminate analysis upon encountering the first processable code segment. When contracts contain multiple `.call` statements, with vulnerabilities present only in subsequent calls, these methods may fail to detect genuine threats.

Action 4 addresses this limitation by splitting singular withdrawal operations across multiple `.call.value` instructions, as demonstrated in Fig. 9. The transformation divides original transfer amounts equally between two separate calls, with the first call designed to be vulnerability-free while the second introduces potential risks. Detection models that prematurely terminate analysis after examining the first call may overlook vulnerabilities in subsequent operations.

The implementation identifies and removes original `.call` statements along with associated `require` checks and balance

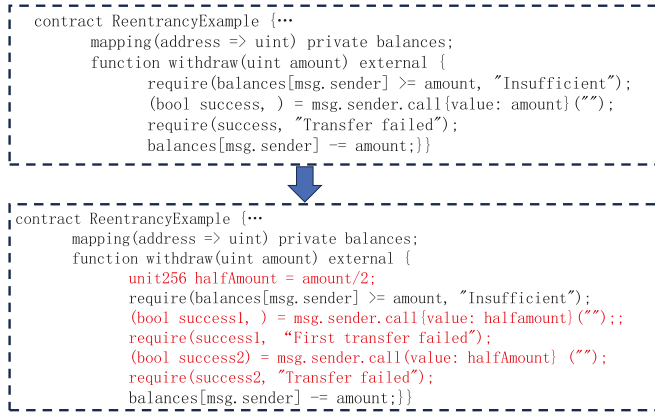


Fig. 9. Multiple functions example.

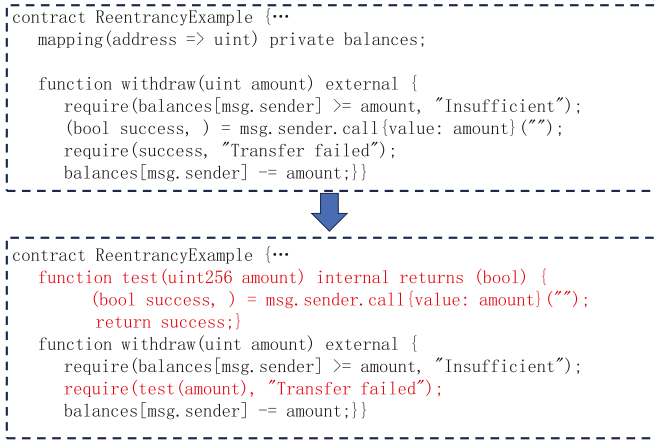


Fig. 10. Read-Only reentrancy example.

updates, replacing them with predefined multi-call implementations that preserve overall contract functionality.

Action 5. Read-Only Hijack: Recent years have witnessed the emergence of novel reentrancy variants, including RORVs that manipulate contract state visibility during external operations. Unlike classical reentrancy, RORVs exploit temporary inconsistencies in read-only contexts (e.g., view functions or `require` statements). The vulnerable contract performs external calls within seemingly read-only code paths, unexpectedly triggering callbacks that exploit state assumptions before updates occur, thereby bypassing intended restrictions [43], [44].

As illustrated in Fig. 10, Action 5 encapsulates `.call.value` operations within “read-only” functions (e.g., `test`) that are invoked within `require` statements. This enables attackers to trigger callbacks during validation phases, establishing reentrancy before state updates complete—characteristic of RORV exploitation patterns.

The implementation closely follows Action 1’s cross-function transformation, with the key distinction that original `.call` statements are replaced with `require` invocations of the newly generated test functions, rather than direct function calls.

C. Grammar Verification

To ensure the practical utility of generated adversarial examples for smart contract RV detection, SCAEG incorporates strict validation principles and verification procedures.

All adversarial transformations must adhere to two fundamental principles: (1) preserving original contract functionality, and (2) maintaining compliance with Solidity syntax rules and code structures. Each generated sample undergoes automated grammar verification to ensure it compiles successfully against target Solidity versions. Samples failing compilation checks are automatically discarded, guaranteeing that only syntactically valid examples are retained for subsequent evaluation.

D. Limitations

While SCAEG succeeds in properly exploiting the model’s blindness using the cross-function and cross-contract approaches for the construction of the adversarial variants, several existing deficiencies remain. First, since the model uses transformation actions that are predefined with the goal of maintaining the program’s semantics, the space of actions does not cover the all real-world mutation strategies. As a result, certain rare or highly domain-specific reentrancy patterns may not be triggered by the current design. Second, the analysis phase, which is tasked with the construction of the call graphs and data flow constraints, must provide precise results. However, when edge cases, like the use of opaque library functions, inline assembly, or proxy-contract structure, are not considered, it is likely that the implementation of SCAEG will not provide useful results. Finally, despite the fact that the model incorporates CFRVs and RORVs, the model’s construction complexity is proportionally higher for larger contracts.

V. DESIGN OF VOTERD

This section presents the design of VoteRD, our ensemble detector designed for robustness against adversarial examples. The design is structured in two phases. First, Section V-A elaborates on the preprocessing and extraction of three complementary feature types: semantic, rule-based, and positional features. Then, Section V-B details the vulnerability detection phase, which employs a multi-model ensemble architecture tailored for these feature types and a final majority voting mechanism for robust prediction.

A. Preprocessing and Feature Extraction

1) Semantic Feature Extraction: Given that low-level `call` operations serve as primary indicators of RVs, our approach begins by locating these critical instructions through control flow analysis. The function containing the target `call` operation is designated as `functionA`. We extract all modifiers associated with `functionA` to capture potential access control patterns. To address CFRVs, we additionally identify two related functions: `functionB` (which invokes `functionA`) and `functionC` (which shares critical state variables with `functionA`). These components are consolidated into an initial program slice, denoted as `sliceA`.

TABLE II
RV DETECTION RULES IN VOTERD

Rules	Description
Rule 1	Checks for reassignment of identical variables before/after low-level <code>call</code> operations, a pattern frequently exploited by reentrancy attacks. Note that CFRVs may persist even when this rule is satisfied.
Rule 2	Verifies whether function callers are subject to restrictions, such as whitelist requirements or reputation-based access controls.
Rule 3	Detects usage of hard-coded addresses, which may prevent certain external attacks by avoiding dynamic address resolution.
Rule 4	Examines whether external calls incorporate gas limitations, a common reentrancy mitigation strategy.
Rule 5	Identifies variables shared across multiple functions, specifically targeting CFRV detection.
Rule 6	Assesses the presence of modifiers in functions, recognizing that while modifiers often indicate protection mechanisms, they may be implemented incorrectly.

Following initial slice construction, we tokenize the line containing the low-level `call` operation, storing the resulting tokens in array M . A secondary slicing operation is then performed: for each line in $sliceA$, we retain only those lines containing tokens from M , with explicit exceptions for lines containing `require` statements or boolean variables, as these often implement crucial access restrictions or locking mechanisms. This refinement produces $sliceB$, a more focused representation that preserves vulnerability-relevant context while eliminating extraneous code. Finally, $sliceB$ is vectorized using Word2Vec to generate semantic feature representations.

2) *Rule-Based Feature Extraction*: While recent research predominantly emphasizes deep learning approaches for RV detection, traditional machine learning methods offer complementary advantages, particularly in interpretability and handling of structured data. The opaque decision-making processes of neural networks make it difficult to ascertain whether predictions genuinely reflect vulnerability patterns or merely capitalize on parameter randomization. To address this limitation, VoteRD incorporates rule-based features derived from established reentrancy mitigation patterns that commonly induce false positives in pure learning-based approaches.

We summarize the six handcrafted rules employed by VoteRD in Table II. These rules function as supplementary features rather than standalone detection mechanisms, complementing other feature types to enhance overall detection capability. Given the limited rule set, we employ one-hot encoding for feature representation. For each contract sample, we systematically traverse the code to identify matches with each predefined rule, constructing a binary rule vector where each position indicates the presence (1) or absence (0) of the corresponding rule pattern.

Notably, these six rules are deliberately designed to be compact and to encode invariant reentrancy semantics (e.g., CEI violations, cross-function state sharing) rather than version-specific Solidity syntax. This design ensures that the rule set remains lightweight and easy to maintain as the language evolves, avoiding the scalability issues of traditional rule-based detection systems.

3) *Positional Feature Extraction*: VoteRD introduces positional features, which capture critical spatial relationships often

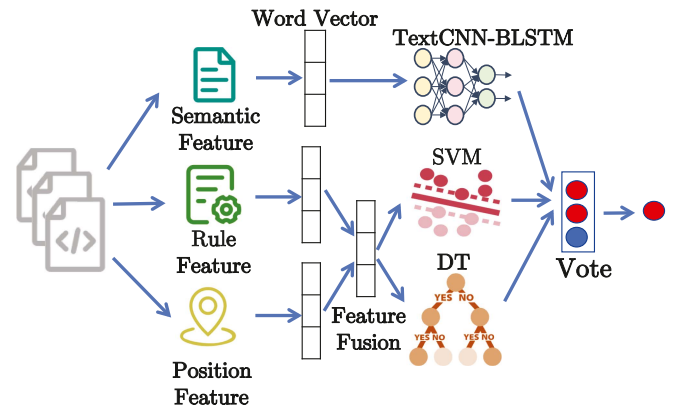


Fig. 11. VoteRD prediction workflow integrating multiple feature types and models.

overlooked in existing studies. The fundamental cause of RVs lies in the improper sequencing of state updates relative to external calls, specifically, positioning balance modifications after low-level `call` operations. Consequently, the relative positioning of these elements provides crucial vulnerability indicators.

To extract positional features, smart contracts are first compiled using solcx for dual purposes: (1) verifying syntactic correctness and (2) generating ASTs. From the AST, we locate functions containing `call` operations and record their positional information. We subsequently identify and count assignment operations, tracking their relative positions to call instructions. To facilitate integration with rule-based features, the resulting positional vectors are zero-padded to fixed dimensions and normalized to the range $[-1, 1]$, mitigating the impact of value disparities across features.

B. Vulnerability Detection

The vulnerability detection phase of VoteRD, illustrated in Fig. 11, employs a multi-model ensemble architecture with an explicit majority voting mechanism. The framework comprises **three distinct model components**:

- *Semantic Feature Model (TextCNN-BLSTM)*: Processes the extracted semantic features. This hybrid architecture combines TextCNN for local syntactic pattern extraction and Bidirectional LSTM (BLSTM) for capturing bidirectional contextual dependencies, enabling comprehensive modeling of program semantics.
- *Rule-based and Positional Feature Model (SVM)*: Processes the combined rule-based and positional features using a SVM classifier, which demonstrates strong performance on structured, low-dimensional data.
- *Rule-based and Positional Feature Model (Decision Tree)*: Processes the same structured features using a Decision Tree (DT) classifier, providing an alternative modeling approach that complements the SVM.

For the textCNN-BLSTM, first, textCNN is used to extract local semantic features, and then BLSTM is employed to capture bidirectional contextual dependencies to obtain global information. This design also brings an additional benefit, namely improving the training efficiency of the model.

For the DT and SVM, which are the highly interpretable model, can explicitly reveal the decision-making process, making the vulnerability-detection results easier to understand. This is particularly important for security analysts and developers, as they need to understand how vulnerabilities are identified and how they can be resolved.

These three model components are trained independently to specialize in their respective feature domains. During the prediction phase, each component produces a binary classification (vulnerable/not vulnerable). The final detection result is determined through **majority voting** among the three models. This ensemble strategy enhances robustness by mitigating individual model limitations while capitalizing on their complementary strengths. The odd number of models used also prevents tie votes.

C. Limitations

Even with the resilience provided by the existence of VoteRD because of the complementarity of BLSTM, TextCNN, SVM, and DT, the ensemble learning paradigm also exhibits several inherent challenges. First, the majority voting strategy used by VoteRD on diverse feature spaces may reduce the potency of deep learning classifiers when shallow classifiers produce errors on complex contract structures. Second, the ensemble learning paradigm ignores the interprocedural execution semantics of the code. Therefore, the ensemble inference phase incurs synchronization overhead since it involves concurrent executions of different models, potentially inducing bottleneck effects during high-speed scanning of the on-chain data.

VI. EXPERIMENT AND RESULT

This section presents a comprehensive evaluation of SCAEG and VoteRD. Section VI-A first describes the construction of our dataset and the evaluation metrics. We then address five Research Questions (RQs) in sequence:

- RQ1 on generalization necessity (Section VI-B): Does RV detection need to consider generalization capability?
- RQ2 on the effectiveness of individual adversarial actions (Section VI-C): How effective are the adversarial actions proposed in SCAEG?
- RQ3 on the combination of actions (Section VI-D): Are combined adversarial actions more effective than single actions?
- RQ4 on the robustness of VoteRD (Section VI-E): Is the integrated model in VoteRD more robust?
- RQ5 on the efficiency of VoteRD (Section VI-F): Does VoteRD introduce acceptable runtime overhead for practical deployment?

Finally, Section VI-G presents ablation studies that validate the contribution of each component in our proposed framework.

A. Dataset and Evaluation Metrics

Our experiments are conducted on a curated dataset of smart contracts designed for reentrancy vulnerability (RV) detection. The initial dataset was constructed by collecting and integrating samples from established public repositories [14], [15], [16],

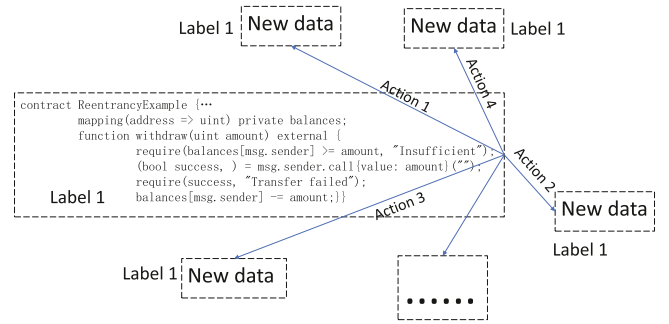


Fig. 12. Data augmentation pipeline utilizing SCAEG.

[17], [45]. After meticulous processing to fix compilation errors, remove duplicates, and exclude anomalous contracts, we obtained a foundational set of 214 compilable and practically meaningful RV samples. To address data scarcity and enhance diversity, we employed our SCAEG framework for adversarial data augmentation, generating additional examples that preserve original vulnerability labels, as illustrated in Fig. 12. This process expanded the final dataset to 1,302 samples, forming a comprehensive benchmark for evaluating detection robustness.

The decision to curate and augment a new dataset was necessitated by the limitations of existing alternatives. Prior datasets [46], [47] are notably small (often containing fewer than 50 samples) and lack structural diversity without modern variants such as CFRV and RORV. Others suffer from significant duplication [21] or are generated artificially by injection tools [48], resulting in redundant and unrealistic vulnerability patterns. Motivated by these observations, our dataset construction intentionally incorporates both cross-function call chains and read-only reentrancy behaviors, enabling comprehensive coverage of contemporary RV types. In particular, RORV samples are integrated through SCAEG's Action 5 (Read-Only Hijack), which encapsulates external calls within view-like contexts to reproduce real-world RORV exploitation patterns. This ensures that the final dataset not only expands in scale but also captures vulnerability categories absent in existing public corpora. Consequently, our dataset provides a more reliable and challenging foundation for training and evaluation.

To assess the effectiveness of SCAEG's adversarial actions and the generated examples, we employ the following metrics:

- *Attack Rate (AR)*: Proportion of generated adversarial examples that successfully induce model misclassification.
- *Valid Rate (VR)*: Proportion of generated examples that preserve original functionality and syntactic validity.

B. RQ1: Generalization Necessity in RV Detection

Most existing RV detection models prioritize accuracy improvements while neglecting generalization capabilities. To evaluate generalization performance, we designed an experiment where models are trained on their authors' original datasets [21], [49] and tested on our SCAEG-generated adversarial dataset. This cross-dataset evaluation protocol, i.e., training each model on its native dataset while testing on a unified, challenging adversarial benchmark, is designed to fairly assess

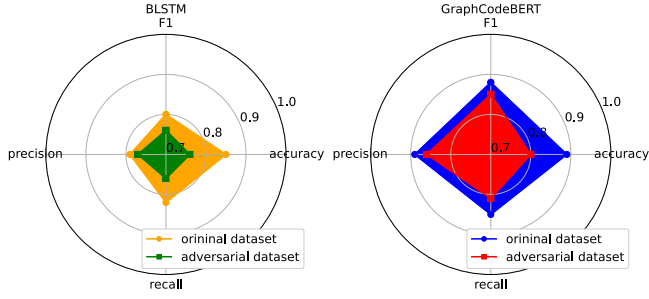


Fig. 13. Performance comparison across original and adversarial datasets.

TABLE III
ADVERSARIAL EFFECTIVENESS ACROSS MODEL TYPES

	BLSTM			GraphCodeBERT		
	AR	VR	Samples*	AR	VR	Samples*
Action 1	42.1%	100%	90	8.4%	100%	18
Action 2	2.8%	100%	6	5.6%	100%	15
Action 3	25.7%	100%	55	1.4%	100%	3
Action 4	41.6%	100%	89	10.2%	100%	22
Action 5	45.3%	100%	97	7.0%	100%	15

*: Number of samples that successfully misled the model.

model robustness against distributional shifts and previously unseen attack patterns.

We evaluated BLSTM and GraphCodeBERT performance on both original and adversarial datasets. As shown in Fig. 13, both models exhibit consistently inferior performance across all metrics (accuracy, precision, recall, F1-score) on the adversarial dataset compared to their original test sets. This performance degradation clearly demonstrates limited generalization capability in existing approaches and underscores the critical importance of generalization consideration in RV detection. Enhancing model generalization requires training on more comprehensive and diverse datasets that better represent real-world variability.

C. RQ2: Effectiveness of SCAEG Adversarial Actions

Current deep learning-based RV detection approaches can be broadly categorized as token-based [21] or graph-based [19], [26], [27], [50], [51]. We selected BLSTM and GraphCodeBERT as representative models from each category. For BLSTM, we replicated the preprocessing pipeline and model configuration from Peng et al. [21], training on their provided dataset. For GraphCodeBERT, we implemented the architecture described in [50], noting its primary focus on vulnerability localization rather than comprehensive detection.

The objective of RQ2 is to evaluate the effectiveness of individual adversarial actions in generating valid adversarial examples, rather than establishing a “no-action” baseline. As outlined in Algorithm 1, this evaluation starts from samples that are correctly classified by the target model, making the baseline performance inherently 100% for the selected sample subset. All adversarial transformations strictly preserve original labels and code semantics. As shown in Table III, SCAEG-generated examples achieve 100% validity rates. The “Samples” column indicates the count of successfully misleading examples.

TABLE IV
PERFORMANCE OF COMBINED ADVERSARIAL ACTIONS

Combination	AR	VR	Samples
Action 1 + Action 2	46.7%	100%	100
Action 1 + Action 3	42.1%	100%	90
Action 1 + Action 4	34.1%	100%	73
Action 2 + Action 3	3.7%	100%	8
Action 2 + Action 4	48.6%	100%	104
Action 2 + Action 5	49.5%	100%	106
Action 3 + Action 4	41.6%	100%	89
Action 3 + Action 5	45.3%	100%	97
Action 4 + Action 5	35.0%	100%	75

Experimental results indicate token-based models demonstrate greater vulnerability to adversarial attacks compared to graph-based approaches. This susceptibility likely stems from token-based models’ tendency to retain redundant information, causing attention diversion to non-predictive features. Among the proposed actions, Actions 1, 4, and 5 prove most effective, targeting complex RV patterns that receive limited attention in current research.

Actions with limited effectiveness, such as Action 2, indicate that most transformed samples remain within model decision boundaries. While suboptimal for adversarial attacks, these samples retain value for data augmentation, as their inclusion in training can strengthen decision boundaries and enhance model robustness.

Notably, graph-based models typically require substantially longer training times due to their inherent architectural complexity. Consequently, subsequent experiments employ token-based approaches for adversarial generation and testing to optimize computational efficiency.

D. RQ3: Effectiveness of Combined Adversarial Actions

We extended our evaluation to pairwise combinations of adversarial actions to investigate their synergistic or antagonistic effects, with results presented in Table IV. The results in Table IV are based on the BLSTM model, as token-based approaches were primarily used for efficiency in combinatorial testing.

The experimental results reveal complex interactions between adversarial actions that transcend simple additive effects. We analyze these patterns through the lens of *perturbation interference* in the model’s feature space, where different transformations may constructively combine or destructively interfere.

The most striking example of *antagonistic interference* occurs in Action 1 + Action 4, which achieves significantly lower AR (34.1%) than either action individually. Both actions fundamentally restructure the `call.value` statement (Action 1 through functional encapsulation and Action 4 through operational decomposition). When applied concurrently, these transformations appear to create conflicting syntactic patterns that neutralize each other’s adversarial potency, potentially moving the sample parallel to rather than across the model’s decision boundary.

Conversely, the combination of Action 2 + Action 5 demonstrates *synergistic enhancement*, achieving the highest AR (49.5%) among all tested pairs. This suggests that perturbations targeting different semantic aspects (modifier insertion and call

TABLE V
PERFORMANCE COMPARISON ON SCAEG-GENERATED VS. ORIGINAL DATASETS

Detection Models	Dataset Generated by SCAEG				Dataset Originally Used in Each Work			
	Accuracy	F1	Precision	Recall	Accuracy	F1	Precision	Recall
Slither [53]	0.5992	0.6081	0.6202	0.5965	0.8990	—	—	—
DR-GCN [19]	0.7497	0.7551	0.9370	0.6323	0.8147	0.8089	0.7236	0.7639
Rechecker [21]	0.7392	0.7540	0.7692	0.7393	0.8847	0.8826	0.8850	0.8848
AWDNN [54]	0.5730	0.0094	0.1538	0.0048	0.8481	0.8604	0.7872	0.9487
SlISE [13]	0.7992	0.3977	0.5645	0.3070	0.8951	0.8000	0.6818	0.9677
VoteRD (Ours)	0.9184	0.9242	0.8714	0.9839	—	—	—	—

TABLE VI
CFRV DETECTION PERFORMANCE COMPARISON

Model	Precision	Recall	F1	Accuracy
VoteRD	0.870	0.870	0.870	0.912
AWDNN [54]	0.037	0.005	0.009	0.514
SlISE [13]	0.667	0.087	0.154	0.812

decomposition) create complementary signals that more effectively mislead the detection model. The orthogonal nature of these transformations likely pushes the sample in a direction that better aligns with the model's vulnerability regions.

Combinations involving Action 3 exhibit more complex behavior. While Action 3 + Action 5 shows modest improvement (45.3%), Action 2 + Action 3 performs poorly (3.7%). This indicates that the effectiveness of redundant code insertion (Action 3) is highly context-dependent, providing marginal benefits when combined with certain transformations but negligible contribution with others, particularly when the primary perturbation is already weak.

These findings challenge the assumption that perturbation stacking necessarily enhances adversarial strength. The efficacy of combined actions depends critically on the semantic alignment and orthogonality of their underlying transformations within the model's feature representation.

We conclude that perturbation stacking does not reliably enhance adversarial attack effectiveness and may even degrade performance. However, these combined-action samples remain valuable for data augmentation [52], as increased variation diversity benefits model robustness and generalization when incorporated into training.

E. RQ4: Robustness of VoteRD Integrated Model

1) *Comparative Evaluation Under Adversarial Conditions:* To ensure a fair comparison under adversarial conditions, all baseline models were evaluated using their original training configurations and datasets as reported in their respective publications. These models were then tested on our SCAEG-generated adversarial examples, with results detailed in Table V. Our method, VoteRD, was trained on our augmented dataset and evaluated on the same adversarial test set, ensuring all methods face equally challenging evaluation conditions. We further conducted focused CFRV evaluation to verify detection coverage for this challenging variant, with comparative results in Table VI.

Adversarial testing reveals consistent performance degradation across most models, with accuracy, F1-score, precision, and

recall metrics declining significantly. These results demonstrate SCAEG's effectiveness in exposing model vulnerabilities and highlight the dataset dependence of existing approaches.

The CFRV-specific evaluation particularly exposes limitations in current detectors: AWDNN and SlISE exhibit critically low recall values (0.005 and 0.087 respectively), indicating near-complete neglect of cross-function vulnerabilities. In contrast, VoteRD maintains balanced performance across all metrics (approximately 0.87), confirming its robust coverage of CFRV patterns frequently overlooked by other approaches.

We attribute these observations to three primary factors:

- Existing preprocessing pipelines typically consider only the first `call` occurrence, ignoring subsequent calls, CFRV patterns, and modifier implementations, thereby failing to handle corresponding adversarial transformations.
- The opaque nature of deep learning models, combined with limited training data, prevents effective learning of comprehensive vulnerability characteristics.
- The architectures of AWDNN and SlISE inherently lack the ability to model cross-function behaviors. AWDNN, as a seq-based classifier, operates strictly at single-function level and can't encode inter-procedural control or data dependencies. Consequently, CFRV cases—whose vulnerability semantics are distributed across multiple functions—are collapsed into benign local patterns and therefore missed. Similarly, SlISE relies on call-site-driven slicing rules and symbolic verification anchored on explicit `.call`. When CFRV attack surfaces are composed through dispersed, weak-dependency operations or non-typical CEI violations, the slicing entry points fail to reach the relevant function chain, causing SlISE to overlook these vulnerabilities. These structural limitations directly explain their near-zero recall under CFRV.

These insights motivated our design of VoteRD, which addresses these limitations through the integrated approach detailed in Section V.

2) *Robustness Evaluation for VoteRD:* To evaluate VoteRD's inherent robustness, we conducted complementary experiments.

First, we compared VoteRD and AWDNN under identical training conditions, i.e., both trained on the same augmented dataset containing CFRV and multiple call patterns, and evaluated them on SCAEG-generated adversarial data. This controlled experiment is designed to isolate the effect of model architecture from that of data augmentation.

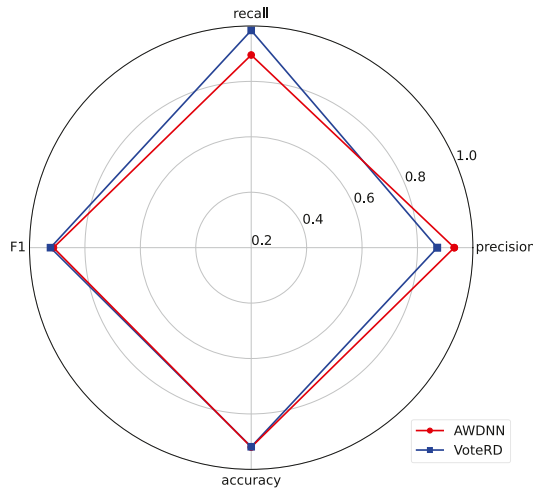


Fig. 14. Performance comparison between VoteRD and AWDNN on adversarial data.

TABLE VII
PERFORMANCE COMPARISON OF VOTERD AND BASELINE METHODS ON EXTERNAL DATASETS

Data	Model	Acc.	Prec.	Recall	F1
AWDNN	AWDNN	0.8481	0.7872	0.9487	0.8604
	VoteRD	0.8228	0.7500	0.9750	0.8478
SmartBugs	SLISE	0.8951	0.6818	0.9677	0.8000
	VoteRD	0.8730	0.8000	0.9655	0.8750

As shown in Fig. 14, under identical training data, the retrained AWDNN achieves Accuracy 0.9198, Precision 0.9328, Recall 0.8952, and F1-score 0.9136, while VoteRD achieves Accuracy 0.9184, Precision 0.8714, Recall 0.9839, and F1-score 0.9242. Two observations are noteworthy: (1) AWDNN benefits substantially from data augmentation, with its recall improving from 0.0048 under its original training configuration (Table V) to 0.8952, confirming the value of SCAEG-generated diverse samples. (2) VoteRD achieves an additional 8.87 percentage point improvement in recall over the retrained AWDNN, demonstrating that the ensemble architecture provides an independent contribution beyond data augmentation alone. Given that vulnerability detection is a security-critical task, higher recall is particularly important, as false negatives (i.e., missed vulnerabilities) may lead to severe security risks. The superior recall of VoteRD thus indicates that its multi-feature ensemble design captures vulnerability-relevant signals that a single-model architecture, even when trained on the same data, may overlook.

Second, to assess whether VoteRD generalizes beyond SCAEG-specific perturbations, we evaluated it on two external, non-SCAEG datasets: the SmartBugs dataset [15] and the reentrancy-related samples provided in the public AWDNN implementation [54]. We chose these datasets because SmartBugs is used in SLISE for evaluation, while the AWDNN codebase provides another publicly available reentrancy dataset that was independently constructed without any involvement of SCAEG. As shown in Table VII, on the AWDNN dataset, VoteRD achieved a higher recall of 0.9750 than AWDNN (0.9487),

TABLE VIII
EFFICIENCY COMPARISON OF DIFFERENT MODELS

Model	Preprocessing (ms)	Inference (ms)	Total (ms)
VoteRD	108.293	43.860	152.153
TextCNN-BLSTM	92.166	38.545	130.710
AWDNN	34.859	68.949	103.808

indicating stronger capability in identifying vulnerable contracts under an external data distribution. On the SmartBugs dataset, VoteRD obtained a substantially higher precision of 0.8000 and F1-score of 0.8750 than SLISE (0.6818 and 0.8000, respectively), while maintaining a comparable recall. These results provide evidence that VoteRD's detection capability is not confined to SCAEG-specific perturbation patterns and generalizes effectively to independently curated reentrancy datasets.

F. RQ5: Efficiency of VoteRD Integrated Model

To assess the practical efficiency of VoteRD, we conducted a runtime comparison under identical experimental conditions, measuring preprocessing time, inference time, and total latency per sample for VoteRD, TextCNN-BLSTM (the semantic feature model within VoteRD, described in Section V-B), and AWDNN. We randomly selected 30 samples from the SmartBugs dataset [15] that were excluded from the training data of both VoteRD and AWDNN, ensuring a fair evaluation on unseen contracts.

As shown in Table VIII, VoteRD requires 152.153 ms per sample in total, compared to 130.710 ms for TextCNN-BLSTM and 103.808 ms for AWDNN. The preprocessing overhead of VoteRD (108.293 ms) is moderately higher than that of TextCNN-BLSTM (92.166 ms), which is expected because VoteRD additionally extracts rule-based and positional features beyond the semantic tokenization shared with TextCNN-BLSTM. However, this overhead is a one-time cost incurred during the feature extraction phase per contract and remains modest in absolute terms.

Notably, although VoteRD incurs a higher total runtime than AWDNN and TextCNN-BLSTM, it still maintains an acceptable inference cost while delivering comparable or superior detection performance, as demonstrated in Section VI-E (Table V) and Section VI-E (Table VII). Although TextCNN-BLSTM is slightly faster than VoteRD (by 21.413 ms per sample), our ablation study (Section VI-G) confirms that the full VoteRD model with rule-based and positional features achieves substantially higher recall and F1-score, indicating that the performance gains from the ensemble architecture outweigh its modest additional runtime cost.

These results demonstrate that VoteRD's ensemble design does not introduce prohibitive latency and remains well-suited for practical pre-deployment contract auditing, where contracts are analyzed individually and a per-contract latency well under 200 ms is entirely acceptable.

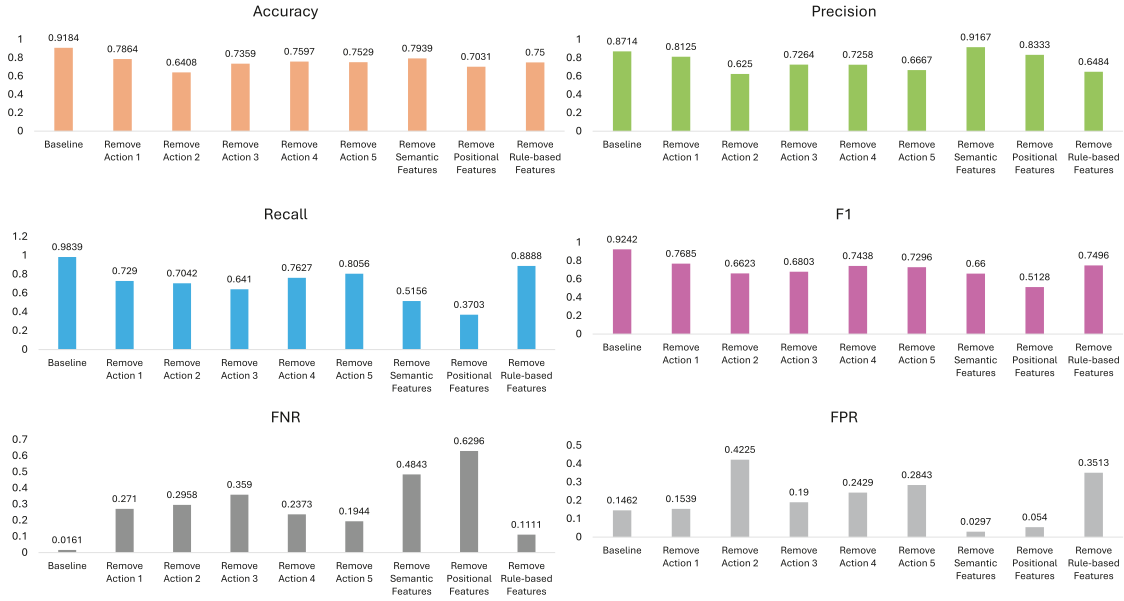


Fig. 15. Ablation Study: SCAEG (Action-level) and VoteRD (Feature-level).

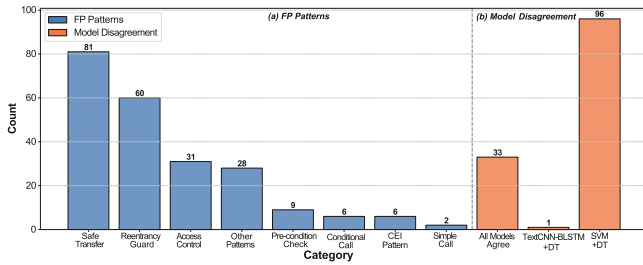


Fig. 16. False positive analysis of VoteRD. (a) Pattern distribution of false positives. (b) Model disagreement among the three ensemble branches.

G. Ablation Studies

We conducted comprehensive ablation studies to validate design choices in both SCAEG and VoteRD, with results summarized in Fig 15. These experiments demonstrate that both the diverse adversarial examples generated by SCAEG and VoteRD’s multi-feature fusion mechanism are indispensable components, whose synergistic interaction ensures robust reentrancy detection against diverse adversarial variants.

For adversarial action ablations, removing Action 2 or Action 3 produces the most significant performance degradation, with substantial declines in Recall and F1 accompanied by increased FPR and FNR rates. This indicates their crucial role in driving robustness improvements. Removing Action 1 or Action 4 causes moderate performance declines: Action 1 removal substantially reduces Recall while only slightly increasing FPR, suggesting its primary function in improving vulnerability coverage; Action 4 removal produces modest across-the-board degradation, highlighting its supplementary role in multi-call scenarios. Notably, Action 5 removal sharply reduces Precision and increases FPR while maintaining relatively high Recall, demonstrating its independent role in false positive control and decision boundary stabilization. Collectively, Actions 2-3 provide strong adversarial perturbations, Actions 1 and 4 expand cross-function and

multi-call coverage, and Action 5 enhances robustness while mitigating false alarms, together forming the foundation of our adversarial generation and robustness enhancement framework.

For feature ablation studies, removing semantic features substantially reduces Recall, excluding positional features drastically lowers Recall, and eliminating rule-based features significantly degrades Precision while increasing FPR. These observations align with our design rationale: semantic features primarily improve vulnerability coverage, positional features capture fundamental reentrancy patterns, and rule-based features help suppress false positives and stabilize predictions.

H. False Positive Case Study

To further understand the limitations of VoteRD, we conducted a detailed analysis of its false positives from two perspectives: vulnerability pattern distribution and model disagreement among the ensemble branches. The results are presented in Fig. 16.

Pattern distribution: We manually categorized all false positive cases into eight patterns based on their structural characteristics. “Safe Transfer” denotes contracts using conservative transfer operations such as `send`; “Reentrancy Guard” covers contracts with explicit anti-reentrancy protection, e.g., lock-based mechanisms; “Access Control” refers to privilege-restricted calls such as `onlyOwner`; “Pre-condition Check” denotes explicit checks before external calls; “Conditional Call” covers branch-controlled calls such as `require`; “CEI Pattern” denotes CEI-like cases where checks-effects-interactions ordering is followed; “Simple Call” refers to structurally simple external-call cases; and “Other Patterns” captures the remaining cases.

As shown in Fig. 16(a), “Safe Transfer” is the most frequent category with 81 cases, followed by “Reentrancy Guard” (60 cases) and “Access Control” (31 cases). This indicates that the majority of false positives occur in contracts that already

incorporate defensive mechanisms, rather than in completely unprotected vulnerable-like code. “Pre-condition Check”, “Conditional Call”, and “CEI Pattern” also account for a non-negligible number of false positives, suggesting that VoteRD can overestimate risk when protected contracts exhibit external-call structures that superficially resemble vulnerable patterns.

Model disagreement. We further examined which branches of the ensemble contribute most to false positive predictions. As shown in Fig. 16(b), the “SVM+DT” combination accounts for 96 cases, whereas only 33 false positives are jointly misclassified by all three branches (“All Models Agree”) and only 1 case is jointly produced by “TextCNN-BLSTM+DT”. This indicates that false positives are primarily driven by the rule-based (SVM) and positional (DT) feature branches, while the semantic branch (TextCNN-BLSTM) helps filter out a substantial portion of these errors.

These findings suggest a clear direction for future improvement: enhancing the rule-based and positional models’ ability to recognize defensive mechanisms (e.g., by incorporating explicit patterns for common anti-reentrancy constructs) could reduce false positives without sacrificing the high recall that the current ensemble achieves.

VII. CONCLUSION

This work presents a comprehensive framework for advancing reentrancy vulnerability detection in smart contracts through adversarial example generation and robust model design. Our contributions address several critical limitations in current research and practical applications:

First, we introduced SCAEG, the first adversarial example generation framework specifically tailored for smart contract RV detection. By developing five novel adversarial actions that preserve code functionality while systematically exposing model vulnerabilities, we demonstrated how state-of-the-art detection systems can be effectively compromised by carefully crafted inputs. Experimental results revealed significant performance degradation in existing models, with AWDNN’s accuracy dropping from 84.81% to 57.30%, corresponding to a reduction of 27.51 percentage points, underscoring the urgent need for more robust detection paradigms.

Second, we addressed the previously overlooked challenge of CFRVs by incorporating specialized patterns into our adversarial generation process. This represents a substantial advancement in vulnerability detection coverage, as conventional tools frequently fail to identify these complex multi-function attack vectors.

Third, leveraging insights from SCAEG’s adversarial examples, we developed VoteRD, an ensemble detection model that integrates multiple feature types with a voting mechanism to achieve superior robustness. Comprehensive evaluation demonstrates that VoteRD significantly outperforms existing approaches, achieving 3.7% higher recall and 7.4% better F1-score while maintaining strong resilience against adversarial attacks.

The implications of this research extend beyond academic contributions to practical security applications:

- Current vulnerability detection systems necessitate rigorous adversarial testing prior to deployment.
- Dataset diversity, particularly inclusion of cross-function patterns, is essential for model generalization.
- Ensemble approaches incorporating multiple feature types present promising directions for robust smart contract analysis.

Future research directions include expanding SCAEG to support multi-contract interaction analysis, investigating adversarial training techniques for enhanced detection robustness, and developing more sophisticated feature representations for CFRVs. The tools and datasets developed in this research are publicly available to support continued advancement in blockchain security. This study not only advances the state-of-the-art in smart contract vulnerability detection but also establishes a new paradigm for evaluating and enhancing security analysis tool robustness through systematic adversarial example generation.

REFERENCES

- [1] P. Sharma, R. Jindal, and M. D. Borah, “A review of smart contract-based platforms, applications, and challenges,” *Cluster Comput.*, vol. 26, no. 1, pp. 395–421, 2023.
- [2] J. Chen et al., “DefectChecker: Automated smart contract defect detection by analyzing EVM bytecode,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2189–2207, Jul. 2022.
- [3] P. Daian, “Analysis of the DAO exploit,” *Hacking, Distrib.*, vol. 6, 2016. Accessed: May 28, 2026. [Online]. Available: <https://incidentdatabase.ai/reports/902/>
- [4] Beosin, “2024 Q3 global web3 security report, AML analysis & crypto regulatory landscape,” Accessed: Apr. 2025. [Online]. Available: https://beosin.com/resources/2024_Q3_Global_Web3_Security_Report_AML_Analysis_%26_Crypto_Regulatory_Landscape.pdf
- [5] Hacken, “H1 2025 web3 security report,” 2025. Accessed: Dec08, 2025. [Online]. Available: <https://hacken.io/insights/h1-2025-security-report/>
- [6] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, “Making smart contracts smarter,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 254–269.
- [7] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, “Securify: Practical security analysis of smart contracts,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 67–82.
- [8] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “SmartCheck: Static analysis of ethereum smart contracts,” in *Proc. Int. workshop Emerg. trends Softw. Eng. Blockchain*, 2018, pp. 9–16.
- [9] B. Jiang, Y. Liu, and W. KwongChan, “ContractFuzzer: Fuzzing smart contracts for vulnerability detection,” in *Proc. ACM/IEEE Int. Conf. Automated Softw. Eng.*, 2018, pp. 259–269.
- [10] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. Tran Minh, “sFuzz: An efficient adaptive fuzzer for solidity smart contracts,” in *Proc. ACM/IEEE Int. Conf. Softw. Eng.*, 2020, pp. 778–788.
- [11] C. De Baets, B. Suleiman, A. Chitizadeh, and I. Razzak, “Vulnerability detection in smart contracts: A comprehensive survey,” 2024, *arXiv:2407.07922*.
- [12] F. Luo et al., “SCVHUNTER: Smart contract vulnerability detection based on heterogeneous graph attention network,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
- [13] Z. Wang, J. Chen, Y. Wang, Y. Zhang, W. Zhang, and Z. Zheng, “Efficiently detecting reentrancy vulnerabilities in complex smart contracts,” *ACM Softw. Eng.*, vol. 1, pp. 161–181, 2024.
- [14] InPlusLab, “Reentrancystudy-data,” 2024. Accessed: Apr. 2025. [Online]. Available: <https://github.com/InPlusLab/ReentrancyStudy-Data>,
- [15] SmartBugs, “Smartbugs-curated,” 2024. Accessed: Apr. 2025. [Online]. Available: <https://github.com/smartbugs/smartbugs-curated>,
- [16] Messi-Q, “Rechecker,” 2024. Accessed: Apr. 2025. [Online]. Available: https://github.com/Messi-Q/ReChecker/tree/master/train_data
- [17] MRdoulestar, “Deescvhunter,” 2024. Accessed: Apr. 2025. [Online]. Available: <https://github.com/MRdoulestar/DeeSCVHunter>

- [18] M. Dhawan, "Analyzing safety of smart contracts," in *Proc. Network Distributed Syst. Secur. Symp.*, pp. 1–15, 2018.
- [19] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural networks," in *Proc. Int. Conf. Int. Joint Conferences Artif. Intell.*, 2021, pp. 3283–3290.
- [20] Z. Wang, J. Chen, P. Zheng, Y. Zhang, W. Zhang, and Z. Zheng, "Unity is strength: Enhancing precision in reentrancy vulnerability detection of smart contract analysis tools," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 1–13, Jan. 2025.
- [21] P. Qian, Z. Liu, Q. HeR. Zimmermann, and X. Wang, "Towards automated reentrancy detection for smart contracts based on sequential models," *IEEE Access*, vol. 8, pp. 19685–19695, 2020.
- [22] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
- [23] A. Søgaard and Y. Goldberg, "Deep multi-task learning with low level tasks supervised at lower layers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 231–235.
- [24] E. Balci, T. Aydede, G. Yilmaz, and E. G. Soyak, "Examining the effectiveness of transformer-based smart contract vulnerability scan," *J. Syst. Softw.*, Art. no. vol231112593, 2025.
- [25] L. Zhang et al., "A novel smart contract reentrancy vulnerability detection model based on bigas," *J. Signal Process. Syst.*, vol. 96, pp. 215–237, 2024.
- [26] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
- [27] Z. Liu, P. Qian, X. Wang, L. Zhu, Q. He, and S. Ji, "Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion," 2021, *arXiv:2106.09282*.
- [28] L. Guo, H. Huang, L. Zhao, P. Wang, S. Jiang, and C. Su, "Reentrancy vulnerability detection based on graph convolutional networks and expert patterns under subspace mapping," *Comput. Secur.*, vol. 142, Art. no. 103894, 2024.
- [29] J. Yang, S. Liu, S. Dai, Y. Fang, K. Xie, and Y. Lu, "ByteEye: A smart contract vulnerability detection framework at bytecode level with graph neural networks," *Automated Softw. Eng.*, vol. 33, no. 1, pp. 1–38, 2026.
- [30] Q. Song, H. Huang, X. Jia, Y. Xie, and J. Cao, "Silence false alarms: Identifying anti-reentrancy patterns on Ethereum to refine smart contract reentrancy detection," in *Proc. Netw. Distrib. System Secur. Symp.*, 2025, pp. 1–18.
- [31] J. Zhang et al., "SmartReco: Detecting read-only reentrancy via fine-grained cross-dapp analysis," in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, 2025, pp. 2138–2150.
- [32] Y. Wu et al., "AdvScanner: Generating adversarial smart contracts to exploit reentrancy vulnerabilities using LLM and static analysis," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 1019–1031.
- [33] X. Li et al., "ScamsWeeper: Detecting illegal accounts in web3 scams via transactions analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 21, pp. 91–104, 2026.
- [34] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [35] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, and Z. Jin, "Generating adversarial examples for holding robustness of source code processing models," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 1169–1176.
- [36] J. Tian, C. Wang, Z. Li, and Y. Wen, "Generating adversarial examples of source code classification models via Q-learning-based markov decision process," in *Proc. IEEE Int. Conf. Softw. Qual. Rel. Secur.*, 2021, pp. 807–818.
- [37] C. Ferretti and M. Saletta, "Deceiving neural source code classifiers: Finding adversarial examples with grammatical evolution," in *Proc. Genet. Evol. Computation Conf. Companion*, 2021, pp. 1889–1897.
- [38] A. Jha and C. K. Reddy, "CodeAttack: Code-based adversarial attacks for pre-trained programming language models," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 14892–14900.
- [39] Y. Yang, H. Fan, C. Lin, Q. Li, Z. Zhao, and C. Shen, "Exploiting the adversarial example vulnerability of transfer learning of source code," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5880–5894, 2024.
- [40] S. Ren et al., "LookAhead: Preventing defi attacks via unveiling adversarial contracts," *ACM Softw. Eng.*, vol. 2, no. FSE, pp. 1847–1869, 2025.
- [41] H. Lakadawala, K. Dzigbede, and Y. Chen, "Detecting reentrancy vulnerability in smart contracts using graph convolution networks," in *Proc. IEEE Consum. Commun. Netw. Conf.*, 2024, pp. 188–193.
- [42] H. Ra, K. Jang, and I. Jang, "Graph attention network with LSTM for smart contract vulnerability detection," in *Proc. IEEE Conf. Pervasive Intell. Comput.*, 2024, pp. 167–172.
- [43] Z. Wang, J. Chen, P. Zheng, Y. Zhang, W. Zhang, and Z. Zheng, "Unity is strength: Enhancing precision in reentrancy vulnerability detection of smart contract analysis tools," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 1–13, Jan. 2025.
- [44] J. Zhang et al., "Smartreco: Detecting read-only reentrancy via fine-grained cross-dapp analysis," 2024, *arXiv:2409.18468*.
- [45] Dependable Systems Lab, "Solidifi," 2024. Accessed: Apr. 2025. [Online]. Available: <https://github.com/DependableSystemsLab/SolidifiBenchmark>
- [46] Z. Zheng, N. Zhang, J. Su, Z. Zhong, M. Ye, and J. Chen, "Turn the rudder: A beacon of reentrancy detection for smart contracts on ethereum," in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, 2023, pp. 295–306.
- [47] T. Durieux, J. F. R. FerreiraAbreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proc. ACM/IEEE Int. Conf. Softw. Eng.*, 2020, pp. 530–541.
- [48] A. Ghaleb and K. Pattabiraman, "How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2020, pp. 415–427.
- [49] D. Guo et al., "GraphcodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [50] Z. Zhang et al., "Reentrancy vulnerability detection and localization: A deep learning based two-phase approach," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.
- [51] H. Wu et al., "Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, 2021, pp. 378–389.
- [52] C. Shorten, T. M. Khoshgoftaar, and B. Furht, "Text data augmentation for deep learning," *J. Big Data*, vol. 8, pp. 101–135, 2021.
- [53] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM Int. Workshop Emerg. Trends Softw. Eng. Blockchain*, 2019, pp. 8–15.
- [54] B. SamuelR. OseiHuang, and Z. Ma, "An attention-based wide and deep neural network for reentrancy vulnerability detection in smart contracts," *J. Syst. Softw.*, vol. 223, 2025, Art. no. 112361.